

How to use this workbook:

1. Open a new Jupyter Notebook.
2. For each step, look at the **Exercise**. Try to write the code yourself in a cell.
3. If you get stuck, expand the **Answer & Explanation** dropdown to see how it's done!

Step 1: The Basics (Importing & Creating Arrays)

Before doing anything, you must import the library. By convention, the entire world imports it as `np`.

The Exercise:

1. Import the `numpy` library as `np`.
2. Convert this standard Python list into a 1D NumPy array: `my_list = [10, 20, 30, 40, 50]`
3. Print the array and check its type.

```
# 1. Import the library
import numpy as np

# 2. Define the list and convert it
my_list = [10, 20, 30, 40, 50]
my_array = np.array(my_list)

# 3. Print and check type
print("Your Array:", my_array)
print("Data Type:", type(my_array))
# Output should say: <class 'numpy.ndarray'> ('nd' means n-dimensional)
```

Step 2: Generating Data (Crucial for ML)

In machine learning, we constantly need to generate placeholder data, weights, or grids of numbers. NumPy has built-in shortcuts for this so you don't have to type numbers manually.

The Exercise:

1. Create an array of ten zeros.
2. Create an array of numbers from 0 to 9 using `np.arange()`.
3. Create a 3x3 matrix (a 2D grid) filled completely with the number `1.0`.

```
# 1. Ten zeros
zeros = np.zeros(10)
print("Zeros:", zeros)

# 2. Numbers 0 to 9
sequence = np.arange(10)
```

```
print("Sequence:", sequence)
```

```
# 3. A 3x3 matrix of ones (notice the double parentheses for the shape!)
ones_matrix = np.ones((3, 3))
print("Ones Matrix:\n", ones_matrix)
```

Step 3: Shapes & Reshaping (The Ultimate ML Skill)

Machine learning models are very picky about data dimensions. If a model expects a 2D grid of data and you give it a 1D flat list, it will crash. You *must* know how to change shapes.

The Exercise:

1. Create a flat 1D array of 12 numbers from 0 to 11 using `np.arange(12)`.
2. Check its current shape using `.shape`.
3. Reshape that flat array into a 2D grid with **3 rows and 4 columns**.

```
# 1. Create flat array
flat_array = np.arange(12)
print("Original:", flat_array)
```

```
# 2. Check shape
print("Original Shape:", flat_array.shape) # Output: (12,) meaning 1D
```

```
# 3. Reshape to 3 rows, 4 columns
matrix = flat_array.reshape(3, 4)
print("Reshaped Matrix:\n", matrix)
print("New Shape:", matrix.shape) # Output: (3, 4)
```

Step 4: Indexing & Slicing (Grabbing the Data You Want)

Just like grabbing specific rows or columns from an Excel sheet, you need to grab specific data from your arrays (like grabbing just the "Labels" or just specific "Features").

The Exercise:

Given this 2D matrix:

```
matrix = np.array([[10, 20, 30],
                  [40, 50, 60],
                  [70, 80, 90]])
```

1. Grab the number `50` (Remember: Python starts counting at 0).
2. Grab the entire first row (`[10, 20, 30]`).
3. Grab the entire last column (`[30, 60, 90]`).

```
matrix = np.array([[10, 20, 30],
                  [40, 50, 60],
                  [70, 80, 90]])
```

```
# 1. Grab 50 (It is in Row index 1, Column index 1)
print("Center element:", matrix[1, 1])
```

```
# 2. Grab the first row (Row index 0, ':' means "all columns")
print("First Row:", matrix[0, :])
```

```
# 3. Grab the last column (':' means "all rows", Column index 2)
print("Last Column:", matrix[:, 2])
```

Step 5: Fast Math (Vectorization)

No loops allowed! Let's see how NumPy performs math on millions of numbers instantly.

The Exercise:

1. Create an array containing [1, 2, 3, 4, 5].
2. In one line of code, multiply every number in that array by 10.
3. Create a second array containing [10, 20, 30, 40, 50] and add the two arrays together.

1. Create array

```
arr1 = np.array([1, 2, 3, 4, 5])
```

2. Multiply by 10 element-wise (No loops!)

```
arr1_scaled = arr1 * 10
```

```
print("Scaled Array:", arr1_scaled) # Output: [10, 20, 30, 40, 50]
```

3. Add two arrays together (They add position-by-position)

```
arr2 = np.array([10, 20, 30, 40, 50])
```

```
result = arr1 + arr2
```

```
print("Sum of Arrays:", result) # Output: [11, 22, 33, 44, 55]
```